NTC-KWS: Noise-aware CTC for Robust Keyword Spotting

Yu Xi[1], Haoyu Li[1], Hao Li[2], Jiaqi Guo[2], Xu Li[2], Wen Ding[3], Kai Yu[1†]
[1]*MoE Key Lab of Artificial Intelligence, AI Institute, X-LANCE Lab, Shanghai Jiao Tong University, Shanghai, China*
[2]*AISpeech Ltd, Suzhou, China* [3]*NVIDIA, Shanghai, China*
{yuxi.cs, haoyu.li.cs, kai.yu}@sjtu.edu.cn {hao.li, jiaqi.guo, xu.li}@aispeech.com wend@nvidia.com

***Abstract*—In recent years, there has been a growing interest in designing small-footprint yet effective Connectionist Temporal Classification based keyword spotting (CTC-KWS) systems. They are typically deployed on low-resource computing platforms, where limitations on model size and computational capacity create bottlenecks under complicated acoustic scenarios. Such constraints often result in overfitting and confusion between keywords and background noise, leading to high false alarms. To address these issues, we propose a noise-aware CTC-based KWS (NTC-KWS) framework designed to enhance model robustness in noisy environments, particularly under extremely low signal-to-noise ratios. Our approach introduces two additional noise-modeling wildcard arcs into the training and decoding processes based on weighted finite state transducer (WFST) graphs: self-loop arcs to address noise insertion errors and bypass arcs to handle masking and interference caused by excessive noise. Experiments on clean and noisy Hey Snips show that NTC-KWS outperforms state-of-the-art (SOTA) end-to-end systems and CTC-KWS baselines across various acoustic conditions, with particularly strong performance in low SNR scenarios.**

***Index Terms*—robust keyword spotting, noise-aware CTC, weighted finite state transducer, noise modeling, wildcard paths**

I. Introduction

Keyword spotting (KWS), also referred to as wake word detection (WWD), serves as a vital interface for human-machine interaction [1]–[4]. KWS systems are typically deployed on devices and operate continuously. As application scenarios grow more diverse, improving noise robustness under resource-constrained conditions has become a key research focus in the field of KWS.

Various approaches have been developed to enhance the noise robustness of KWS systems. One approach is to introduce single/multi-channel speech enhancement (SE) modules preceding the KWS module [5]–[10]. Another approach involves designing new training strategies or architectures to achieve noise-robust KWS systems [11]–[13]. Regardless of the methods, simulated noisy data plays a critical role in enhancing KWS performance. Our preliminary experiments highlight two key observations: First, at low signal-to-noise ratio (SNR) levels, strong noise energy naturally leads to lower recall; Second, simulated data with excessively low SNR causes the model to overfit to noise, utimately degrading overall performance. These occur because the noise in challenging training data masks portions of keyword speech, leading the model to confuse noise with the target keyword.

Noise-induced speech errors can lead to mismatches between the speech content and paired text transcripts during training. Previous CTC-based ASR approaches have introduced wildcard modeling training method [14]–[17] to handle transcript inaccuracies. Conversely, this study addresses noise-induced speech errors and proposes the *NTC-KWS* framework, which integrates both noise-aware training and decoding strategies to enhance model robustness in noisy conditions. More specifically, we incorporate two types of noise-modeling wildcard arcs into the CTC training framework to adaptively model noise and further improve the WFST-based decoding search space, boosting KWS performance.

The contributions of this work are summarized as follows:

1) We propose noise-aware CTC training criterion to model noise-dominant speech segments under complex acoustic scenarios to prevent noise overfitting.
2) We are the first to adopt a training-free decoding strategy that incorporates noise-modeling paths into WFST-based KWS decoding. This approach enhances performance under noisy environments without requiring NTC training and offers further improvements when applied within our NTC-KWS framework.
3) Compared to SOTA end-to-end and CTC-KWS baselines, the proposed NTC-KWS achieves superior performance across both clean and noisy conditions, with notable improvements in extremely challenging scenarios.

II. Methodology

A. WFST-based Implementation of CTC

Connectionist temporal classification (CTC) [18] is a widely used loss function for training sequence learning model with length-mismatch of inputs and labels, and is naturally suitable for ASR or KWS. For ASR, an input acoustic feature sequence can be represented as $\mathbf{x} = [x_1, x_2, ..., x_T] \in \mathbb{R}^{T\times D}$, where each x_t is a D-dimensional acoustic feature vector and T represents the total frames. The output label sequence can be represented as $\mathbf{y} = [y_1, y_2, ..., y_U] \in \mathbb{R}^{U\times 1}$, where y_u is the label token and U is the number of total tokens. It's worth noting that T is must larger than U because CTC introduces a special blank token ϕ to model the null output at each frame. The actual alignment sequence for the input is $\boldsymbol{\pi} = [\pi_1, \pi_2, ..., \pi_T] \in \mathbb{R}^{T\times 1}$, where each π_t represents either a normal phoneme or a blank token ϕ in the CTC vocabulary $\mathcal{V}_{ctc}$. The objective of CTC is to minimize the summation of the negative log-likelihood of the alignment sequences given the input sequence. Therefore, the CTC loss can be formulated as:

$$\mathcal{L}_{CTC} = -\log P(\mathbf{y}|\mathbf{x}) = -\log \sum_{\boldsymbol{\pi}\in\mathcal{B}^{-1}(\mathbf{y})} P(\boldsymbol{\pi}|\mathbf{x}), \tag{1}$$

where $\mathcal{B}$ is defined as a mapping from the legal CTC alignment $\boldsymbol{\pi}$ to the label $\mathbf{y}$ and $\mathcal{B}^{-1}$ stands for the inverse mapping. In addition, the decoding is to find the most possible target of the speech. We can formulate it as:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} P(\mathbf{y}|\mathbf{x}) \approx \arg\max_{\mathbf{y},\boldsymbol{\pi}\in\mathcal{B}^{-1}(\mathbf{y})} P(\boldsymbol{\pi}|\mathbf{x}), \tag{2}$$

where the approximate derivation introduces the Viterbi decoding to find the most possible alignments.

Weighted finite state transducer (WFST) [19], aiming to map an input sequence to an output one, provides an efficient implementation

† Corresponding Author.

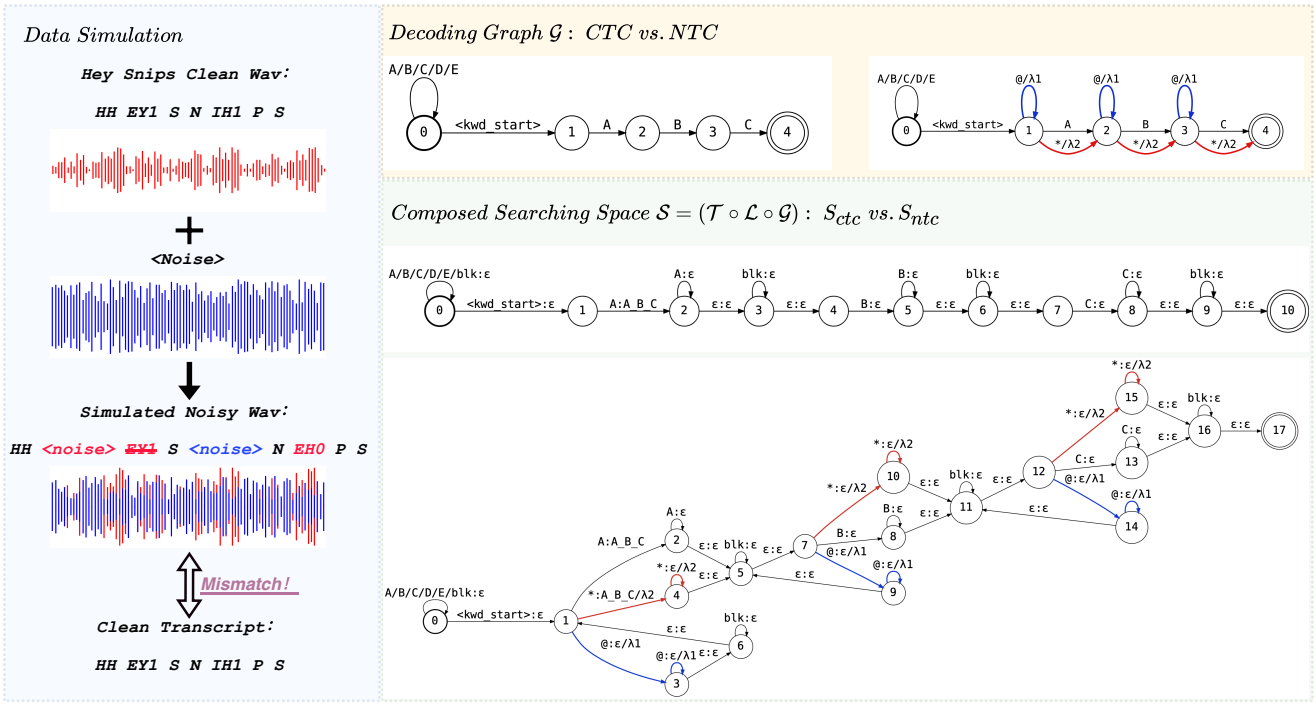

Fig. 1. The overview illustrates noisy data simulation and comparisons of CTC and NTC WFST-based decoding graphs. In the left part, we show three types of errors by a noise simulation example introduced by overwhelming noise: masking and interference (shown in red), and insertion (shown in blue). In the right part, we provide a toy example where the keyword is set to "A B C". The decoding transition rules under grammar and token levels are represented by $\mathcal{G}$ and $\mathcal{S}$, respectively. In NTC, we highlight two types of additional wildcard arcs in red and blue, corresponding to the error colors of the simulated example shown on the left. In the composed graphs $\mathcal{S}$, λ_1 and λ_2 represent the wildcard transition costs. ϵ to the left of : denotes an empty transition, while on the right, it indicates a null output.

for the CTC training and decoding. Specifically, the searching space $\mathcal{S}$ is defined by the composition operations of $\mathcal{T}$, $\mathcal{L}$ and $\mathcal{G}$, which represent CTC merging (remove ϕ and merge repeated tokens), lexicon (a mapping from word to tokens) and grammar FSTs, respectively. In addition, the emission dense graph $\mathcal{E}$ is built to model acoustic posteriors. So we can reformulate the CTC training and decoding as follows:

$$\mathcal{L}_{CTC} = -Forward(\ \mathcal{E}(\mathbf{x}) \cap (\ \mathcal{T} \circ \mathcal{L} \circ \mathcal{G}(\mathbf{y})\ )\ ), \tag{3}$$

and

$$\hat{\mathbf{y}} = Viterbi(\ \mathcal{E}(\mathbf{x}) \cap (\ \mathcal{T} \circ \mathcal{L} \circ \mathcal{G}\ )\ ). \tag{4}$$

Here, $\cap$ and $\circ$ denote the intersection and composition operations on WFSTs, respectively. $\mathcal{G}(\mathbf{y})$ represents a linear FST constructed for the transcript $\mathbf{y}$. The terms *Forward* and *Viterbi* refer to the forward computation and Viterbi decoding on the composed WFST graph. Specifically, *Token Passing* is an efficient method for implementing Viterbi decoding within the WFST framework. When a token reaches the final node, a decoding path is considered complete.

B. CTC-based Keyword Spotting

Compared to the CTC-based ASR described in Section II-A, KWS follows a nearly identical training procedure, with only two minor differences in the decoding phase.

Decoding graph. The grammar graph $\mathcal{G}$ of the KWS [20]–[23] is more compact and lightweight. It consists of only keyword paths and an optional background path rather than the whole acoustic space.

Confidence score measure. Search results are transformed into keyword activation scores by computing confidence scores of speech segments. Efforts have been made to evaluate different strategies for computing confidence scores for CTC-based KWS [21], [22]. In this work, we follow the confidence metric proposed in [22].

C. Training and Decoding of NTC-KWS

Although CTC-based KWS performs well in typical acoustic conditions, its performance degrades considerably when speech segments are overwhelmed by noise. As shown on the left side of Figure 1, excessive noise introduces errors in the simulated speech, including content masking, noise artifacts, and pronunciation distortion. This results in a mismatch between the speech and the transcript, increasing the possibility of noise-induced false triggers. To enhance noise robustness, we propose NTC-KWS, which introduces two types of wildcard arcs: self-loop arcs and bypass arcs during both training and decoding. These arcs represent distinct token-level graph errors in $\mathcal{G}$, as shown in Figure 1. When parts of the keyword are obscured by noise, self-loop arcs (@) model insertion errors in keyword labels (denoted by blue errors and arcs). In scenarios where phonemes are distorted or masked by noise, bypass arcs ($\star$) account for substitution or deletion errors (depicted as red errors and arcs). These wildcards effectively capture the concept of "**excessive speech noise and interference**". The acoustic posteriors for $\star$ and @ arcs, representing **the average posteriors across all speech**, are computed as follows:

$$P(\pi_i(@)|x_i) = P(\pi_i(\star)|x_i) = \frac{\sum_{j \in \mathcal{V}_{ctc}/\{\phi\}} P(\pi_i(j)|x_i)}{len(\mathcal{V}_{ctc}/\{\phi\})}, \tag{5}$$

where $\pi_i(j)$ denotes the predicted token j in vocabulary $\mathcal{V}_{ctc}$ at frame x_i. To ensure that the model prioritizes learning the correct alignments over the wildcard paths, large initial penalties are assigned

to the wildcard arcs. These penalties decay exponentially with respect to the training epochs:

$$\lambda_{@}^{(l)} = \lambda_{@}^{(0)} * \beta_{@}^{l}, \quad \lambda_{\star}^{(l)} = \lambda_{\star}^{(0)} * \beta_{\star}^{l}, \tag{6}$$

where l-th denotes the number of the current epoch, and $\lambda_{@}^{(0)}$, $\beta_{@}$, $\lambda_{\star}^{(0)}$, $\beta_{\star}$ are hyper-parameters, set empirically.

Although the model has learned to handle noise-dominant parts of the keyword using wildcard paths through NTC-based training, the standard CTC-KWS graph for decoding fails to fully leverage the enhanced training, especially in low SNR scenarios. To better alignment the training and decoding procedure, we must also expand the decoding search space with wildcard paths by modifying the grammar graph $\mathcal{G}$, as shown in the right part of Figure 1. For example, assuming the keyword is "A B C" and the vocabulary $\mathcal{V}_{ctc}$ includes the tokens "A B C D E" along with the blank token ϕ, we introduce self-loop (@) and bypass ($\star$) arcs between adjacent sub-words of the "A B C" keyword sequence on graph $\mathcal{G}$ to model potential errors accordingly. The NTC grammar graph $\mathcal{G}_{ntc}$ is then composed with the transition $\mathcal{T}$ and lexicon $\mathcal{L}$ to create the NTC-KWS search space $\mathcal{S}_{ntc}$. Compared to the CTC-KWS search space $\mathcal{S}_{ctc}$, $\mathcal{S}_{ntc}$ offers alternative noise-modeling paths for token passing, which improves decoding topology for noisy KWS in the following aspects. First, the alignment of training and decoding methods ensures consistency. Second, recognizing every token in the keyword becomes challenging when noise is predominant. Therefore, loosening decoding restrictions in complex acoustic environments is reasonable to boost recall, as it gives the model the option to select wildcard arcs against noise. To balance recall and false alarms, we introduce transition costs $\lambda_{@}$ and $\lambda_{\star}$ (λ_1 and λ_2 in Figure 1) for the two types of arcs in the NTC decoding graph, respectively. However, if a decoding path consists solely of wildcard tokens without containing any keyword phonemes, that path will be discarded.

III. Experimental Setup

A. Datasets

To assess the performance and noise robustness of the systems, we construct datasets using the following open-source corpora.

- **LibriSpeech (LS)** [24]: LibriSpeech is an open-source, widely used ASR corpus containing 960 hours of English speech with corresponding transcripts. The dataset is used as the pre-training dataset for the base model and general ASR data for KWS models.
- **Hey Snips (Snips)** [25]: Hey-Snips is an open-source dataset for KWS centered around the keyword "Hey Snips." Due to the absence of transcripts for negative utterances, the negative data is used exclusively for evaluation and not for training. We combined all negative data from the train, dev, and test sets to create a large test set, following the data preparation approach in [12]. The duration of the new negative test set is about 97 hours.
- **WHAM!** [26]: The WHAM! dataset consists of various types of ambient noise, including music, restaurant sounds, etc. We incorporate this dataset to synthesize noisy utterances for both training and evaluation.

Training. We pre-train the base speech encoder using clean LibriSpeech. Then both Snips and an equivalent amount of LibriSpeech are utilized to fine-tune a KWS model. To simulate noisy dataset, each clean waveform from Snips and LS is mixed with a randomly selected noise sample from WHAM!, with the SNR chosen from a uniform distribution ranging from 0 to 20 dB. The clean and noisy speech are gathered to construct the training dataset.

Evaluation. We simulate noisy negative test set in the same manner as the training setting and simulate test positive speech at different fixed SNR levels. All positive test samples from the Snips dataset are mixed with noise at SNR levels of $\{0, 5, 10, 15, 20\}$ dB. To further evaluate robustness, we simulate an additional dataset under -5 dB, representing an extremely low and out-of-domain SNR.

B. Configuration

Models. We leverage the toolkit K2 [27] to build the differential WFSTs for training and leverge OpenFST [28] to build decoding systems. The Deep Feedforward Sequential Memory Network (DFSMN) [29], a lightweight yet effective speech encoder, is a mainstream architecture for KWS tasks [12], [30], [31]. We use this architecture as the backbones for both CTC and NTC KWS systems. All models are initialized from the same pre-trained checkpoint and optimized by SGD [32]. The DFSMN architecture consists of 6 layers, with hidden and projection sizes set to 512 and 320, respectively. Each DFSMN layer has left and right context orders of 8 and 2. The final output unit of the CTC-based model includes 70 monophones derived from the CMU Pronouncing Dictionary "cmudict-0.7b" [33], along with the special blank symbol ϕ. Additionally, the NTC includes up to two extra special tokens: the self-loop token @ and the bypass token $\star$.

Acoustic features. We extract 40-dimensional log Mel-filter bank coefficients (FBank) from the raw audio using a 25ms window and a 10ms hop. Additionally, we employ two data augmentation strategies: online speech perturbation [34], with a warping factor uniformly sampled from $\{0.9, 1.0, 1.1\}$, and SpecAugment [35], using 2 frequency masks with a maximum $F = 10$ and 2 time masks with a maximum $T = 50$ for each utterance. We concatenate the current frame with 5 preceding and 5 succeeding frames to create the model input. We also apply a frame-skipping factor of 3 to reduce computational cost through down-sampling.

Hyper-parameters. Decay coefficients $\beta_{@}$ and $\beta_{\star}$ are set to 0.999 and 0.975, respectively. The initial wildcard costs $\lambda_{@}^{(0)}$ and $\lambda_{\star}^{(0)}$ are both set to -4 for training. Decoding coefficients $\lambda_{@}$ and $\lambda_{\star}$ are tuned during test. There are at most 20 active tokens during token passing inference for both CTC and NTC systems.

C. Evaluation Details

Baselines. We construct comprehensive baselines to evaluate our NTC-KWS system. Firstly, we compare the CTC-based model with the state-of-the-art (SOTA) end-to-end (E2E) systems for the Hey Snips dataset, which is open-sourced in WeKws [36], to prove the KWS models constructed by CTC or its variants can get superior performance. Then we further conduct detailed comparison among NTC-KWS and CTC-related baselines at different in-domain and out-of-domain SNRs to evaluate the improved training and decoding methods.

Evaluation metrics. We report recall values at various SNR levels for a specific false alarm rate (FAR). Published SOTA models typically present recalls at FAR levels of 0.5 or 1 per hour. To ensure a fair comparison, we adopt the same settings when benchmarking against these systems. However, this evaluation standard is relatively lenient, as performance tends to be strong at such FAR levels. To minimize the impact of deviations and randomness, we assess all systems under stricter conditions. Therefore, after the initial comparison, we present recall at FAR = 0.05 per hour.

TABLE I
THE COMPARISONS OF E2E BASELINES AND OUR CTC AND NTC KWS SYSTEMS ON THE CLEAN HEY SNIPS DATASET. "POS. SNIPS" REFERS TO THE POSITIVE PORTION OF THE SNIPS DATASET, WHILE "EQU. LS" INDICATES THAT WE ADD AN EQUAL AMOUNT OF LIBRISPEECH UTTERANCES AS GENERAL ASR DATA.

ID	System	Training Data	Test Neg. Duration	FAR		
				0.05	0.5	1.0
A	RIL KWS [37]	Official Snips	23hrs	-	96.47	97.18
B	WaveNet [38]			-	99.88	-
C1	WeKws-MDTC [36]			-	99.88	99.92
C2	WeKws-MDTC	Pos. Snips + Equ. LS	97hrs	89.52	98.85	99.29
D	CTC-KWS			98.77	**99.72**	**99.76**
E	NTC-KWS			**98.97**	99.64	99.72

IV. RESULTS AND ANALYSIS

A. The Comparison with E2E SOTA Baselines

The upper part of the table shows the results using the official Snips data, while the bottom part reports results based on reconstructed data due to the lack of transcriptions for the Snips negative samples. This reconstruction is necessary since CTC-based models cannot be trained on speech data without transcripts, as discussed in Section III-A. We compare our CTC and NTC models with SOTA E2E baselines on the clean Hey Snips dataset at FAR of 0.5 or 1.0 per hour. Our models can achieve comparable performance (D&E vs. B or C1). When models are trained on the same data, the results in the bottom section of Table I show that both CTC and NTC models outperform the SOTA baseline at FARs of 0.5 or 1.0 per hour (D&E vs. C2). Additionally, under the more challenging condition of FAR = 0.05 per hour, CTC and NTC models exhibit an absolute recall improvement of over 9%. The results in Table I further demonstrate that our models surpass the baselines, especially under strict test conditions, validating the effectiveness of the CTC-KWS baseline and proposed NTC-KWS.

B. The Impact of NTC Training and Decoding

In this section, we examine the individual effects of NTC training and decoding. Table II illustrates the performance across different combinations of training and inference strategies for CTC and NTC. Given the CTC inference (F vs. D), updating the training criterion to NTC alone does not improve performance, as NTC training does not compel the model to treat noise as part of the speech signal. As a result, model F behaves similarly to a CTC model trained on clean, high-quality speech. Training the model with CTC loss and testing it using the NTC decoding method (D vs. G) yields a 2.7% absolute improvement in average recall. This gain is particularly pronounced at low SNR levels (SNR = -5, 0), highlighting the challenge of token decoding under such conditions. Our proposed decoding strategy relaxes constraints in complex environments, leading to significant performance gains. This suggests that if retraining the original CTC KWS model is not feasible but performance improvements in noisy settings are required, adopting the NTC decoding strategy offers a practical solution. Furthermore, applying NTC decoding to the NTC model results in a 2.2% absolute increase in average recall (E vs. G), primarily due to the alignment between the training and inference processes. This improvement highlights the benefits of consistency between NTC training and decoding. In summary, the proposed NTC-KWS framework achieves a 4.9% absolute improvement over the CTC-KWS baseline (E vs. D). Notably, there is a significant performance gain of 15% and 8.3% at low SNRs of -5 and 0, respectively, underscoring the robustness of NTC-KWS in extremely noisy conditions.

TABLE II
THE EVALUATION OF THE DIFFERENT COMBINATIONS FOR CTC-KWS AND NTC-KWS. "TRAIN" REFERS TO THE SPECIFIC TRAINING CRITERION EMPLOYED DURING THE MODEL TRAINING PHASE, WHILE "TEST" INDICATES THE TYPE OF DECODING GRAPH USED DURING THE INFERENCE PHASE.

ID	Train	Test	SNR						Avg.
			-5	0	5	10	15	20	
D	CTC	CTC	41.6	75.1	90.2	95.8	97.5	98.3	83.1
F	NTC	CTC	41.7	74.3	89.8	95.8	97.5	98.1	82.9
G	CTC	NTC	52.0	79.8	91.7	96.2	97.4	98.0	85.8
E	NTC	NTC	**56.6**	**83.4**	**93.2**	**97.6**	**98.2**	**98.7**	**88.0**

C. Search for Optimal Decoding Configuration

This section further investigates the importance of two hyper-parameters in the decoding configuration. Grid search is performed on the values of $\lambda_{@}$ and $\lambda_{\star}$. The best performance is achieved when both $\lambda_{@}$ and $\lambda_{\star}$ are set to positive values, indicating that the NTC model does not overfit to shortcuts during NTC training and still requires boosting paths of wildcards during inference. We also experimented with setting the decoding transition coefficients to negative values. While the improvement is less pronounced than with positive values, it still outperforms the CTC-KWS baseline. When both coefficients are set to $-\infty$, the NTC-KWS model behaves like model F in Table II, indicating that the NTC decoding strategy collapses into the CTC approach. Another finding from Table III shows that the decoding phase is more sensitive to $\lambda_{\star}$ than $\lambda_{@}$, and setting $\lambda_{\star}$ too high cannot be feasible. In future work, we plan to conduct experiments across various noise types to further investigate these decoding parameters.

TABLE III
DIFFERENT CONFIGURATIONS OF SELF-LOOP WEIGHT $\lambda_{@}$ AND BYPASS WEIGHT $\lambda_{\star}$ DURING INFERENCE. NEGATIVE NUMBERS MEAN PUNISHMENT, WHILE POSITIVE NUMBERS MEAN ENCOURAGEMENT.

$\lambda_{@}$	$\lambda_{\star}$	SNR						Avg.
		-5	0	5	10	15	20	
4	2	56.6	83.4	93.2	97.6	98.2	98.7	88.0
4	4	40.0	70.9	88.8	94.9	96.9	97.4	81.5
4	0	55.2	82.3	92.8	97.2	97.9	98.4	87.3
4	$-\infty$	55.0	82.1	92.8	96.9	97.9	98.3	87.2
2	2	50.3	81.7	92.4	97.0	98.1	98.7	86.4
0	2	49.5	80.9	92.4	96.9	98.0	98.6	86.0
$-\infty$	2	49.3	80.9	92.4	96.8	98.1	98.7	86.0

V. CONCLUSION

In this paper, we present NTC-KWS, a variant of the CTC-based KWS framework tailored for noisy environments, particularly in extremely low SNR conditions. By incorporating various types of wildcards during both the training and decoding phases, we reduce the risk of model overfitting to noise. Modeling noise-dominant segments as wildcard tokens enables the model to better distinguish keyword speech from background noise. Our system achieves SOTA performance on the clean Hey Snips dataset compared to previous E2E systems. Specifically, the NTC-KWS framework demonstrates an absolute improvement of 4.9% in average recall across all SNR levels over the standard CTC-KWS system. In low SNR conditions, such as -5 dB and 0 dB, it achieves absolute recall gains of 15% and 8.3%, respectively. These improvements underscore the robustness of the proposed system in challenging, noisy environments.

REFERENCES

[1] G. Chen, C. Parada, and G. Heigold, "Small-footprint keyword spotting using deep neural networks," in *Proc. IEEE ICASSP*, 2014, pp. 4087–4091.

[2] R. Alvarez and H.-J. Park, "End-to-end streaming keyword spotting," in *Proc. IEEE ICASSP*, 2019, pp. 6336–6340.

[3] Y. Wang, H. Lv, D. Povey, L. Xie, and S. Khudanpur, "Wake word detection with streaming transformers," in *Proc. IEEE ICASSP*, 2021, pp. 5864–5868.

[4] I. López-Espejo, Z.-H. Tan, J. H. L. Hansen, and J. Jensen, "Deep spoken keyword spotting: An overview," *IEEE Access*, pp. 4169–4199, 2022.

[5] M. Yu, X. Ji, Y. Gao, L. Chen, J. Chen, J. Zheng, D. Su, and D. Yu, "Text-dependent speech enhancement for small-footprint robust keyword detection," in *Proc. Interspeech*, 2018, pp. 2613–2617.

[6] M. Yu, X. Ji, B. Wu, D. Su, and D. Yu, "End-to-end multi-look keyword spotting," in *Proc. Interspeech*, 2020, pp. 66–70.

[7] Y. Gu, Z. Du, H. Zhang, and X. Zhang, "An efficient joint training framework for robust small-footprint keyword spotting," in *International Conference on Neural Information Processing*, 2020, pp. 12–23.

[8] Y. Na, Z. Wang, L. Wang, and Q. Fu, "Joint ego-noise suppression and keyword spotting on sweeping robots," in *Proc. IEEE ICASSP*. IEEE, 2022, pp. 7547–7551.

[9] C. Yang, Y. M. Saidutta, R. S. Srinivasa, C.-H. Lee, Y. Shen, and H. Jin, "Robust Keyword Spotting for Noisy Environments by Leveraging Speech Enhancement and Speech Presence Probability," in *Proc. Interspeech*, 2023, pp. 1638–1642.

[10] H. Li, B. Yang, Y. Xi, L. Yu, T. Tan, H. Li, and K. Yu, "Text-aware speech separation for multi-talker keyword spotting," in *Proc. Interspeech*, 2024, pp. 337–341.

[11] I. López-Espejo, Z. Tan, and J. Jensen, "A novel loss function and training strategy for noise-robust keyword spotting," *IEEE ACM Trans. Audio Speech Lang. Process.*, pp. 2254–2266, 2021.

[12] Y. Xi, H. Li, B. Yang, H. Li, H. Xu, and K. Yu, "TDT-KWS: Fast and accurate keyword spotting using token-and-duration transducer," in *Proc. IEEE ICASSP*, 2024, pp. 11 351–11 355.

[13] H. Zhu, X. Wang, K. Wang, and H. Zhan, "Temporal convolution shrinkage network for keyword spotting," in *Proc. IEEE ICASSP*, 2024.

[14] X. Cai, J. Yuan, Y. Bian, G. Xun, J. Huang, and K. Church, "W-CTC: a connectionist temporal classification loss with wild cards," in *International Conference on Learning Representations*, 2022.

[15] V. Pratap, A. Hannun, G. Synnaeve, and R. Collobert, "Star temporal classification: Sequence modeling with partially labeled data," in *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

[16] D. Gao, M. Wiesner, H. Xu, L. P. García, D. Povey, and S. Khudanpur, "Bypass temporal classification: Weakly supervised automatic speech recognition with imperfect transcripts," in *Proc. Interspeech*, 2023, pp. 924–928.

[17] D. Gao, H. Xu, D. Raj, L. P. García-Perera, D. Povey, and S. Khudanpur, "Learning from flawed data: Weakly supervised automatic speech recognition," in *Proc. IEEE ASRU*, 2023, pp. 1–8.

[18] A. Graves, S. Fernández, F. J. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proc. ICML*, 2006, pp. 369–376.

[19] M. Mohri, F. Pereira, and M. Riley, "Weighted finite-state transducers in speech recognition," *Computer Speech & Language*, vol. 16, no. 1, pp. 69–88, 2002.

[20] M. Sun, D. Snyder, Y. Gao, V. K. Nagaraja, M. Rodehorst, S. Panchapagesan, N. Strom, S. Matsoukas, and S. Vitaladevuni, "Compressed time delay neural network for small-footprint keyword spotting." in *Proc. Interspeech*, 2017, pp. 3607–3611.

[21] Y. He, R. Prabhavalkar, K. Rao, W. Li, A. Bakhtin, and I. McGraw, "Streaming small-footprint keyword spotting using sequence-to-sequence models," in *Proc. IEEE ASRU*, 2017, pp. 474–481.

[22] T. Bluche, M. Primet, and T. Gisselbrecht, "Small-footprint open-vocabulary keyword spotting with quantized LSTM networks," *CoRR*, vol. abs/2002.10851, 2020. [Online]. Available: https://arxiv.org/abs/2002.10851

[23] Y. Xi, B. Yang, H. Li, J. Guo, and K. Yu, "Contrastive learning with audio discrimination for customizable keyword spotting in continuous speech," in *Proc. IEEE ICASSP*, 2024, pp. 11 666–11 670.

[24] V. Panayotov *et al.*, "Librispeech: an asr corpus based on public domain audio books," in *Proc. IEEE ICASSP*, 2015, pp. 5206–5210.

[25] A. Coucke, M. Chlieh, T. Gisselbrecht, D. Leroy, M. Poumeyrol, and T. Lavril, "Efficient keyword spotting using dilated convolutions and gating," in *Proc. IEEE ICASSP*, 2019, pp. 6351–6355.

[26] G. Wichern, J. Antognini, M. Flynn, L. R. Zhu, E. McQuinn, D. Crow, E. Manilow, and J. L. Roux, "Wham!: Extending speech separation to noisy environments," in *Proc. Interspeech*, 2019, pp. 1368–1372.

[27] "k2 toolkit," https://github.com/k2-fsa/k2.

[28] M. Riley, C. Allauzen, and M. Jansche, "Openfst: An open-source, weighted finite-state transducer library and its applications to speech and language," in *Human Language Technologies: Conference of NAACL*. The Association for Computational Linguistics, 2009, pp. 9–10.

[29] S. Zhang, M. Lei, Z. Yan, and L. Dai, "Deep-FSMN for large vocabulary continuous speech recognition," in *Proc. IEEE ICASSP*, 2018, pp. 5869–5873.

[30] Y. Zhang, S. Sun, and L. Ma, "Tiny transducer: A highly-efficient speech recognition model on edge devices," in *Proc. IEEE ICASSP*, 2021, pp. 6024–6028.

[31] Y. Tian, H. Yao, M. Cai, Y. Liu, and Z. Ma, "Improving rnn transducer modeling for small-footprint keyword spotting," in *Proc. IEEE ICASSP*, 2021, pp. 5624–5628.

[32] S. Ruder, "An overview of gradient descent optimization algorithms," *ArXiv*, vol. abs/1609.04747, 2016.

[33] "The CMU pronouncing dictionary," http://www.speech.cs.cmu.edu/cgi-bin/cmudict.

[34] T. Ko, V. Peddinti, D. Povey, and S. Khudanpur, "Audio augmentation for speech recognition," in *Proc. Interspeech*, 2015.

[35] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "Specaugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019.

[36] J. Wang, M. Xu, J. Hou, B. Zhang, X. Zhang, L. Xie, and F. Pan, "Wekws: A production first small-footprint end-to-end keyword spotting toolkit," in *Proc. IEEE ICASSP*. IEEE, 2023, pp. 1–5.

[37] K. Zhang, Z. Wu, D. Yuan, J. Luan, J. Jia, H. Meng, and B. Song, "Re-Weighted Interval Loss for Handling Data Imbalance Problem of End-to-End Keyword Spotting," in *Proc. Interspeech*, 2020, pp. 2567–2571.

[38] A. Coucke, M. Chlieh, T. Gisselbrecht, D. Leroy, M. Poumeyrol, and T. Lavril, "Efficient keyword spotting using dilated convolutions and gating," in *Proc. IEEE ICASSP*, 2019, pp. 6351–6355.